

廈門大學



信息学院软件工程系

《计算机网络》实验报告

题 目 实验三 PCAP 监听和分析

班 级 数字媒体技术 2023 级数媒班

姓 名 马鑫

学 号 37220232203780

实验时间 2025 年 10 月 23 日

2025 年 2 月 15 日

填写说明

- 1、本文件为 Word 模板文件，建议使用 Microsoft Word 2024 打开，在可填写的区域中如实填写；
- 2、填表时勿改变字体字号，保持排版工整，打印为 PDF 文件提交；
- 3、文件总大小尽量控制在 1MB 以下，最大勿超过 5MB；
- 4、在实验课结束 14 天内，按实验报告提交到我校课程网站的指定位置，源代码等主要材料上传在公开的代码托管平台上。
- 5、鼓励同学之间探讨，鼓励合理使用人工智能平台，提升效率，但不应滥用相关资源，如抄袭代码和代写作业。

1 实验目的

通过完成实验，理解数据链路层、网络层、传输层和应用层的基本原理。掌握用 Wireshark 观察网络流量并辅助网络侦听相关的编程；掌握用 Libpcap 或 WinPcap 库侦听并处理以太网帧和 IP 报文的方法；熟悉以太网帧、IP 报文、TCP 段和 FTP 命令的格式概念，掌握 TCP 协议的基本机制；熟悉帧头部或 IP 报文头部各字段的含义。熟悉 TCP 段和 FTP 数据协议的概念，熟悉段头部各字段和 FTP 控制命令的指令和数据的含义。

2 实验环境

Windows11, C++

3 实验结果

1、用侦听解析软件观察数据格式

用 Wireshark 或 Omnipcap 等网络侦听软件网络上的数据流，验证理论课讲授的网络协议层次嵌套，验证帧格式、IP 报文格式、TCP 段格式和 FTP 协议命令和响应的格式，验证 MAC 地址、IP 地址、TCP 端口等协议地址格式。

在 Wireshark 中输出了每一个数据包的如下几部分

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	42.186.111.153	10.32.44.202	TLSv1.2	86	Application Data
2	0.000149	10.32.44.202	42.186.111.153	TLSv1.2	90	Application Data
3	0.102362	42.186.111.153	10.32.44.202	TCP	60	443 → 51656 [ACK] Seq=33 Ack=37 Win=83 Len=0
4	0.102417	42.186.111.153	10.32.44.202	TLSv1.2	89	Application Data
5	0.102552	10.32.44.202	42.186.111.153	TLSv1.2	93	Application Data
6	0.122683	10.32.44.202	36.150.102.201	TLSv1.2	191	Application Data
7	0.204800	36.150.102.201	10.32.44.202	TLSv1.2	185	Application Data
8	0.204868	42.186.111.153	10.32.44.202	TCP	60	443 → 51656 [ACK] Seq=68 Ack=76 Win=83 Len=0
9	0.245869	10.32.44.202	36.150.102.201	TCP	54	57802 → 7826 [ACK] Seq=138 Ack=132 Win=511 Len=0
10	2.457703	fe80::42fe:95ff:feff::ff02::1		ICMPv6	118	Router Advertisement from 40:fe:95:fe:80:01
11	5.632036	fe80::42fe:95ff:feff::ff02::1		ICMPv6	118	Router Advertisement from 40:fe:95:fe:80:01
12	7.829916	10.32.44.202	20.189.173.1	TLSv1.2	139	Application Data
13	7.830008	10.32.44.202	20.189.173.1	TLSv1.2	918	Application Data, Application Data
14	8.000699	20.189.173.1	10.32.44.202	TCP	60	443 → 63049 [ACK] Seq=1 Ack=950 Win=16388 Len=0
15	8.000923	20.189.173.1	10.32.44.202	TLSv1.2	93	Application Data
16	8.137978	10.32.44.202	20.189.173.1	TCP	54	63049 → 443 [ACK] Seq=950 Ack=40 Win=510 Len=0
17	8.396993	20.189.173.1	10.32.44.202	TLSv1.2	154	Application Data
18	8.397901	10.32.44.202	20.189.173.1	TLSv1.2	89	Application Data
19	8.703990	20.189.173.1	10.32.44.202	TCP	60	443 → 63049 [ACK] Seq=140 Ack=985 Win=16388 Len=0
20	8.908832	fe80::42fe:95ff:feff::ff02::1		ICMPv6	118	Router Advertisement from 40:fe:95:fe:80:01
21	9.139313	10.32.44.202	36.141.40.31	TCP	55	63231 → 16616 [ACK] Seq=1 Ack=1 Win=514 Len=1
22	9.216090	36.141.40.31	10.32.44.202	TCP	66	16616 → 63231 [ACK] Seq=1 Ack=2 Win=501 Len=0 SLE=1 SRE=2
23	9.217631	10.32.44.202	36.141.40.31	TCP	55	63235 → 16616 [ACK] Seq=1 Ack=1 Win=514 Len=1
24	9.318763	36.141.40.31	10.32.44.202	TCP	66	16616 → 63235 [ACK] Seq=1 Ack=2 Win=501 Len=0 SLE=1 SRE=2
25	9.420890	36.150.102.201	10.32.44.202	TCP	60	7826 → 57914 [ACK] Seq=1 Ack=1 Win=21 Len=0


```

> Frame 1: Packet, 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF...
> Ethernet II, Src: Net3Techno-fe80:01 (40:fe:95:fe:80:01), Dst: LiteonTechno-ee:2c:4f (f4:6a:dd:ee:2c:4f)
> Internet Protocol Version 4, Src: 42.186.111.153, Dst: 10.32.44.202
> Transmission Control Protocol, Src Port: 443, Dst Port: 51656, Seq: 1, Ack: 1, Len: 32
> Transport Layer Security
0000  f4 6a dd ee 2c 4f 40 fe 95 fe 80 01 08 00 45 04  j...@.....E
0010  00 48 05 6a 40 00 32 06 72 05 2a ba 6f 99 0a 20  H:8:2...o
0020  2c ca 01 bb c8 51 9d 32 ba 94 eb 53 dc 50 18  ,...Q:2...S:P
0030  00 51 c9 a9 00 00 17 03 03 00 1b 12 a8 22 7a 41  S.....Pa
0040  bc b8 40 6a 9d 30 f5 95 70 45 19 15 c4 b2 24 d8  .@j0...pE...$
0050  80 76 86 a1 83 6f  v...o

```

选择第三帧验证以上格式

(1) 帧格式

该帧在线路上共有 60bytes，捕获长度 60bytes，到达时间 2025 年 10 月 22 日 13:11:50.153958000 （ 中 国 标 准 时 间 ） ， 接 口 ： \Device\NPF_{3972CAFF-3623-41ED-A856-0A98F5962A73}，封装类型：以太网（Ethernet）

```
✓ Frame 3: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{3972CAFF-3623-41ED-A856-0A98F5962A73}
  Section number: 1
  > Interface id: 0 (\Device\NPF_{3972CAFF-3623-41ED-A856-0A98F5962A73})
    Encapsulation type: Ethernet (1)
    Arrival Time: Oct 22, 2025 13:11:50.153958000 中国标准时间
    UTC Arrival Time: Oct 22, 2025 05:11:50.153958000 UTC
    Epoch Arrival Time: 1761109910.153958000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 102.213000 milliseconds]
    [Time delta from previous displayed frame: 102.213000 milliseconds]
    [Time since reference or first frame: 102.362000 milliseconds]
    Frame Number: 3
    Frame Length: 60 bytes (480 bits)
    Capture Length: 60 bytes (480 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp]
    Character encoding: ASCII (0)
    [Coloring Rule Name: TCP]
    [Coloring Rule String: tcp]
```

(2) IP 报文格式

版本号：IPv4，头部长度的：20bytes，总长度：40bytes，标志：不可被切片，偏移地址：0，生存期：50，类型：TCP，头部校验和：未被校验，源 IP 地址：42.186.111.153，目的地址：10.32.44.202。

```

v Internet Protocol Version 4, Src: 42.186.111.153, Dst: 10.32.44.202
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
v Differentiated Services Field: 0x04 (DSCP: LE, ECN: Not-ECT)
  0000 01.. = Differentiated Services Codepoint: Lower Effort (1)
  .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 40
  Identification: 0x056b (1387)
v 010. .... = Flags: 0x2, Don't fragment
  0... .... = Reserved bit: Not set
  .1.. .... = Don't fragment: Set
  ..0. .... = More fragments: Not set
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 50
  Protocol: TCP (6)
  Header Checksum: 0x7224 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 42.186.111.153
  Destination Address: 10.32.44.202
  [Stream index: 0]

```

(3) TCP 段格式

源端口号：443，目的端口号：51656，滑动窗口大小：83，未使用紧急指针。

```

v Transmission Control Protocol, Src Port: 443, Dst Port: 51656, Seq: 33, Ack: 37, Len: 0
  Source Port: 443
  Destination Port: 51656
  [Stream index: 0]
  [Stream Packet Number: 3]
> [Conversation completeness: Incomplete (12)]
  [TCP Segment Len: 0]
  Sequence Number: 33 (relative sequence number)
  Sequence Number (raw): 1369256666
  [Next Sequence Number: 33 (relative sequence number)]
  Acknowledgment Number: 37 (relative ack number)
  Acknowledgment number (raw): 2498450432
  0101 .... = Header Length: 20 bytes (5)
> Flags: 0x010 (ACK)
  Window: 83
  [Calculated window size: 83]
  [Window size scaling factor: -1 (unknown)]
  Checksum: 0xa55d [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
> [Timestamps]
> [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]

```

(4) FTP 协议命令和响应的格式

第 317 条是一个命令协议，noop 表示无动作，除了来自服务器上的承认。第 325 条是一个响应请求，200 表示命令 OK。

ip.addr==121.192.180.236						
No.	Time	Source	Destination	Protocol	Length	Info
317	32.238460	10.32.44.202	121.192.180.236	FTP	60	Request: noop
318	32.261134	121.192.180.236	10.32.44.202	FTP	73	Response: 200 Command okay.
319	32.261196	10.32.44.202	121.192.180.236	TCP	54	64761 → 21 [ACK] Seq=7 Ack=20 Win=1022 Len=0
320	32.261250	10.32.44.202	121.192.180.236	FTP	70	Request: CWD /  u/
321	32.274281	121.192.180.236	10.32.44.202	FTP	90	Response: 250 Directory changed to /  u
322	32.274348	10.32.44.202	121.192.180.236	TCP	54	64761 → 21 [ACK] Seq=23 Ack=56 Win=1022 Len=0
323	32.274458	10.32.44.202	121.192.180.236	FTP	62	Request: TYPE A
324	32.541974	10.32.44.202	121.192.180.236	TCP	62	[TCP Retransmission] 64761 → 21 [PSH, ACK] Seq=23 Ack=56 Win=1022 Len=0
325	32.596983	121.192.180.236	10.32.44.202	FTP	74	Response: 200 Type set to A.
326	32.596983	121.192.180.236	10.32.44.202	TCP	66	[TCP Dup ACK 325#1] 21 → 64761 [ACK] Seq=76 Ack=31 Win=259 Len=0 SLE=23 SRE=31
327	32.597046	10.32.44.202	121.192.180.236	TCP	54	64761 → 21 [ACK] Seq=31 Ack=76 Win=1022 Len=0
328	32.597248	10.32.44.202	121.192.180.236	FTP	60	Request: PASV
330	32.772619	121.192.180.236	10.32.44.202	FTP	107	Response: 227 Entering Passive Mode (121,192,180,236,226,168)
331	32.772682	10.32.44.202	121.192.180.236	TCP	54	64761 → 21 [ACK] Seq=37 Ack=129 Win=1021 Len=0
332	32.772758	10.32.44.202	121.192.180.236	TCP	66	65012 → 58024 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
333	32.781365	121.192.180.236	10.32.44.202	TCP	66	58024 → 65012 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1386 WS=256 SACK_PERM
334	32.781411	10.32.44.202	121.192.180.236	TCP	54	65012 → 58024 [ACK] Seq=1 Ack=1 Win=262144 Len=0
335	32.781465	10.32.44.202	121.192.180.236	FTP	60	Request: LIST
336	32.804595	121.192.180.236	10.32.44.202	FTP-DA...	1440	FTP Data: 1386 bytes (PASV) (LIST)
337	32.804654	10.32.44.202	121.192.180.236	TCP	54	65012 → 58024 [ACK] Seq=1 Ack=1387 Win=262144 Len=0
338	32.808587	121.192.180.236	10.32.44.202	FTP-DA...	1440	FTP Data: 1386 bytes (PASV) (LIST)
339	32.808641	10.32.44.202	121.192.180.236	TCP	54	65012 → 58024 [ACK] Seq=1 Ack=2773 Win=262144 Len=0
340	32.824085	121.192.180.236	10.32.44.202	FTP	107	Response: 150 Opening ASCII mode data connection for /bin/ls.
341	32.824149	10.32.44.202	121.192.180.236	TCP	54	64761 → 21 [ACK] Seq=43 Ack=182 Win=1021 Len=0
342	32.829152	121.192.180.236	10.32.44.202	FTP-DA...	410	FTP Data: 356 bytes (PASV) (LIST)
343	32.829189	10.32.44.202	121.192.180.236	TCP	54	65012 → 58024 [ACK] Seq=1 Ack=3130 Win=261632 Len=0
344	32.829232	10.32.44.202	121.192.180.236	TCP	54	65012 → 58024 [FIN, ACK] Seq=1 Ack=3130 Win=261632 Len=0
345	32.832407	121.192.180.236	10.32.44.202	TCP	60	58024 → 65012 [ACK] Seq=3130 Ack=2 Win=66304 Len=0
346	32.833384	121.192.180.236	10.32.44.202	FTP	78	Response: 226 Transfer complete.
347	32.833399	10.32.44.202	121.192.180.236	TCP	54	64761 → 21 [ACK] Seq=43 Ack=206 Win=1021 Len=0

(5) 验证 MAC 地址

在抓包数据的以太网格式可以找到本地的 MAC 地址。

```
Ethernet II, Src: NewH3CTechno_fe:80:01 (40:fe:95:fe:80:01), Dst: LiteonTechno_ee:2c:4f
> Destination: LiteonTechno_ee:2c:4f (f4:6a:dd:ee:2c:4f)
> Source: NewH3CTechno_fe:80:01 (40:fe:95:fe:80:01)
  Type: IPv4 (0x0800)
  [Stream index: 0]
  Padding: 000000000000
```

在命令行使用 `ipconfig/all` 可以查看本机的 MAC 地址。

```
无线局域网适配器 WLAN:

   连接特定的 DNS 后缀 . . . . . : xmu.edu.cn
   描述. . . . . : Realtek RTL8852BE WiFi 6 802.11ax PCIe Adapter
   物理地址. . . . . : F4-6A-DD-EE-2C-4F
   DHCP 已启用 . . . . . : 是
   自动配置已启用. . . . . : 是
   IPv6 地址. . . . . : 2001:da8:e800:71e2:62c9:fdce:6e64:b831(首选)
   临时 IPv6 地址. . . . . : 2001:da8:e800:71e2:1ccf:be26:879a:e5a2(首选)
   本地链接 IPv6 地址. . . . . : fe80::e94a:56a6:dd27:26f7%9(首选)
   IPv4 地址. . . . . : 10.32.44.202(首选)
   子网掩码. . . . . : 255.255.224.0
   获得租约的时间. . . . . : 2025年10月22日 8:49:13
   租约过期的时间. . . . . : 2025年10月22日 17:52:00
   默认网关. . . . . : fe80::42fe:95ff:fefe:8001%9
                        10.32.32.1
   DHCP 服务器 . . . . . : 172.18.0.15
   DHCPv6 IAID . . . . . : 99904221
   DHCPv6 客户端 DUID . . . . . : 00-01-00-01-2C-56-33-1F-F4-6A-DD-EE-2C-4F
   DNS 服务器 . . . . . : 210.34.0.18
                        210.34.0.14
   TCP/IP 上的 NetBIOS . . . . . : 已启用
```

经过对比可以发现本机 MAC 地址是抓包数据中的目的地址，代表这条数据是发送给本机的。

(6) 验证 IP 地址

```
Internet Protocol Version 4, Src: 42.186.111.153, Dst: 10.32.44.202
```

```
  0100 .... = Version: 4
```

```
  .... 0101 = Header Length: 20 bytes (5)
```

```
✓ Differentiated Services Field: 0x04 (DSCP: LE, ECN: Not-ECT)
```

```
  0000 01.. = Differentiated Services Codepoint: Lower Effort (1)
```

```
  .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
```

```
Total Length: 40
```

```
Identification: 0x0338 (824)
```

```
✓ 010. .... = Flags: 0x2, Don't fragment
```

```
  0... .... = Reserved bit: Not set
```

```
  .1.. .... = Don't fragment: Set
```

```
  ..0. .... = More fragments: Not set
```

```
...0 0000 0000 0000 = Fragment Offset: 0
```

```
Time to Live: 50
```

```
Protocol: TCP (6)
```

```
Header Checksum: 0x7457 [validation disabled]
```

```
[Header checksum status: Unverified]
```

```
Source Address: 42.186.111.153
```

```
Destination Address: 10.32.44.202
```

```
[Stream index: 1]
```

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . : xmu.edu.cn
描述. . . . . : Realtek RTL8852BE WiFi 6 802.11ax PCIe Adapter
物理地址. . . . . : F4-6A-DD-EE-2C-4F
DHCP 已启用. . . . . : 是
自动配置已启用. . . . . : 是
IPv6 地址. . . . . : 2001:da8:e800:71e2:62c9:fdce:6e64:b831(首选)
临时 IPv6 地址. . . . . : 2001:da8:e800:71e2:1ccf:be26:879a:e5a2(首选)
本地链接 IPv6 地址. . . . . : fe80::e94a:56a6:dd27:26f7%9(首选)
IPv4 地址. . . . . : 10.32.44.202(首选)
子网掩码. . . . . : 255.255.224.0
获得租约的时间. . . . . : 2025年10月22日 8:49:13
租约过期的时间. . . . . : 2025年10月22日 17:52:00
默认网关. . . . . : fe80::42fe:95ff:fefe:8001%9
                    10.32.32.1
DHCP 服务器. . . . . : 172.18.0.15
DHCPv6 IAID. . . . . : 99904221
DHCPv6 客户端 DUID. . . . . : 00-01-00-01-2C-56-33-1F-F4-6A-DD-EE-2C-4F
DNS 服务器. . . . . : 210.34.0.18
                    210.34.0.14
TCP/IP 上的 NetBIOS. . . . . : 已启用
```



美班味小狗·

同理可以验证抓包数据的 IP 目的地址就是本机的 IP 地址。

(7) 验证 TCP 端口

```

Transmission Control Protocol, Src Port: 54332, Dst Port: 16618, Seq: 0, Len: 0
  Source Port: 54332
  Destination Port: 16618
  [Stream index: 15]
  [Stream Packet Number: 1]
  > [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 2733368096
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0x5556 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No
  > [Timestamps]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]

```

通过 Netstat -p TCP 查看 TCP 端口，通过 IP 地址进行定位，与抓取的数据报一致。

```

C:\Users\lenovo>Netstat -p TCP
活动连接

```

协议	本地地址	外部地址	状态
TCP	10.32.44.202:49446	4.145.79.80:https	ESTABLISHED
TCP	10.32.44.202:52502	223.113.135.65:https	CLOSE_WAIT
TCP	10.32.44.202:52503	115.190.141.103:https	CLOSE_WAIT
TCP	10.32.44.202:53046	162.159.192.1:https	ESTABLISHED
TCP	10.32.44.202:53351	20.42.73.24:https	TIME_WAIT
TCP	10.32.44.202:53544	115.190.63.128:https	ESTABLISHED
TCP	10.32.44.202:53591	115.190.127.84:https	ESTABLISHED
TCP	10.32.44.202:53804	39.156.126.202:https	ESTABLISHED
TCP	10.32.44.202:53813	111.48.172.241:https	ESTABLISHED
TCP	10.32.44.202:53822	120.241.130.197:https	ESTABLISHED
TCP	10.32.44.202:53907	36.141.40.65:16618	ESTABLISHED
TCP	10.32.44.202:53937	36.141.40.31:16618	ESTABLISHED
TCP	10.32.44.202:53957	36.141.40.31:16618	ESTABLISHED
TCP	10.32.44.202:53963	36.141.40.31:16618	ESTABLISHED
TCP	10.32.44.202:53990	36.141.40.31:16618	ESTABLISHED
TCP	10.32.44.202:57440	115.190.141.103:https	CLOSE_WAIT

2、用侦听解析软件观察 TCP 机制

用 Wireshark 侦听并观察 TCP 数据段。观察其建立和撤除连接的过程，观察段 ID、窗口机制和拥塞控制机制等。

(1) TCP 三次握手

所用测试服务器 IP 为 121.194.11.72，本机 IP 为 10.27.193.99，

28650	288.834820	10.27.193.99	121.194.11.72	TCP	66	56238 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
28666	288.887688	121.194.11.72	10.27.193.99	TCP	66	443 → 56238 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1460 SACK_PERM WS=2048
28667	288.887743	10.27.193.99	121.194.11.72	TCP	54	56238 → 443 [ACK] Seq=1 Ack=1 Win=131328 Len=0

第一次握手：本机的 TCP 向服务器请求连接报文，首部中的同部位 SYN=1，并选择序号 seq=0

```

Transmission Control Protocol, Src Port: 56238, Dst Port: 443, Seq: 0, Len: 0
  Source Port: 56238
  Destination Port: 443
  [Stream index: 158]
  [Stream Packet Number: 1]
  > [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3548999118
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x002 (SYN)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    .... 0... = Congestion Window Reduced: Not set
    .... 0... = ECN-Echo: Not set
    .... 0... = Urgent: Not set
    .... 0... = Acknowledgment: Not set
    .... 0... = Push: Not set
    .... 0... = Reset: Not set
    > .... 0...1. = Syn: Set
    .... 0...0 = Fin: Not set
  [TCP Flags: .....S.]
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0x04d5 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), N
  > [Timestamps]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]

```

第二次握手：服务器的 TCP 收到连接请求报文段后同意并发回确认，确认报文段中 SYN=1，ACK=1，确认号 ack=1，自己选择的序号 seq=0。

```

Transmission Control Protocol, Src Port: 443, Dst Port: 56238, Seq: 0, Ack: 1, Len: 0
  Source Port: 443
  Destination Port: 56238
  [Stream index: 158]
  [Stream Packet Number: 2]
  > [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 3547150062
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 3548999119
  1000 .... = Header Length: 32 bytes (8)
  v Flags: 0x012 (SYN, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    .... 0... = Congestion Window Reduced: Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    > .... .... ..1. = Syn: Set
    .... .... ...0 = Fin: Not set
    [TCP Flags: .....A..S.]
  Window: 42340
  [Calculated window size: 42340]
  Checksum: 0x4ff1 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted,
  > [Timestamps]
  > [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]

```

第三次握手：本机收到报文段之后再次向服务器确认，ACK=1，确认号 ack=1，序号 seq=1。

```

v Transmission Control Protocol, Src Port: 56238, Dst Port: 443, Seq: 1, Ack: 1, Len: 0
  Source Port: 56238
  Destination Port: 443
  [Stream index: 158]
  [Stream Packet Number: 3]
  > [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3548999119
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 3547150063
  0101 .... = Header Length: 20 bytes (5)
  v Flags: 0x010 (ACK)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    .... 0... = Congestion Window Reduced: Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    .... .... ...0 = Fin: Not set
    [TCP Flags: .....A....]
  Window: 513
  [Calculated window size: 131328]
  [Window size scaling factor: 256]
  Checksum: 0x342b [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > [Timestamps]
  > [SF0/ACK analysis]

```

(2) TCP 四次挥手

32303	349.097121	10.27.193.99	121.194.11.72	TCP	54 56238 → 443 [ACK] Seq=3711 Ack=1534 Win=131328 Len=0
32304	349.097161	121.194.11.72	10.27.193.99	TCP	60 443 → 56238 [FIN, ACK] Seq=1534 Ack=3711 Win=43008 Len=0
32305	349.097170	10.27.193.99	121.194.11.72	TCP	54 56238 → 443 [ACK] Seq=3711 Ack=1535 Win=131328 Len=0
32306	349.097319	10.27.193.99	121.194.11.72	TCP	54 56238 → 443 [FIN, ACK] Seq=3711 Ack=1535 Win=131328 Len=0
32311	349.191320	121.194.11.72	10.27.193.99	TCP	60 443 → 56238 [RST] Seq=1535 Win=0 Len=0

第一次挥手：服务器首先向本机发送链接释放报文段，并停止再发送数据，FIN=1，序号 seq=1534，等待主机确认。

```

v Transmission Control Protocol, Src Port: 443, Dst Port: 56238, Seq: 1534, Ack: 3711, Len: 0
  Source Port: 443
  Destination Port: 56238
  [Stream index: 158]
  [Stream Packet Number: 28]
  > [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 1534    (relative sequence number)
  Sequence Number (raw): 3547151596
  [Next Sequence Number: 1535    (relative sequence number)]
  Acknowledgment Number: 3711    (relative ack number)
  Acknowledgment number (raw): 3549002829
  0101 .... = Header Length: 20 bytes (5)
v Flags: 0x011 (FIN, ACK)
  000. .... = Reserved: Not set
  ...0 .... = Accurate ECN: Not set
  .... 0... = Congestion Window Reduced: Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...1 .... = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
  > .... .... ...1 = Fin: Set
  > [TCP Flags: .....A...F]
Window: 21
[Calculated window size: 43008]
[Window size scaling factor: 2048]
Checksum: 0x219b [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0

```

第二次挥手：主机发出确认 ACK=1，确认号 ack=1535，报文段序号 seq=3711。


```

> Internet Protocol Version 4, Src: 10.27.193.99, Dst: 121.194.11.72
▼ Transmission Control Protocol, Src Port: 56238, Dst Port: 443, Seq: 3711, Ack: 1535, Len: 0
    Source Port: 56238
    Destination Port: 443
    [Stream index: 158]
    [Stream Packet Number: 29]
    > [Conversation completeness: Complete, WITH_DATA (63)]
    [TCP Segment Len: 0]
    Sequence Number: 3711 (relative sequence number)
    Sequence Number (raw): 3549002829
    [Next Sequence Number: 3711 (relative sequence number)]
    Acknowledgment Number: 1535 (relative ack number)
    Acknowledgment number (raw): 3547151597
    0101 .... = Header Length: 20 bytes (5)
    ▼ Flags: 0x010 (ACK)
        000. .... = Reserved: Not set
        ...0 .... = Accurate ECN: Not set
        .... 0... = Congestion Window Reduced: Not set
        .... .0.. = ECN-Echo: Not set
        .... ..0. = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 0... = Push: Not set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        [TCP Flags: .....A....]
    Window: 513
    [Calculated window size: 131328]
    [Window size scaling factor: 256]
    Checksum: 0x1faf [unverified]
    [Checksum Status: Unverified]

```

第三次挥手：主机通知 TCP 释放连接，FIN=1，ACK=1，序号 seq=3711，确认号 ack=1535。

```

v Transmission Control Protocol, Src Port: 56238, Dst Port: 443, Seq: 3711, Ack: 1535, Len: 0
  Source Port: 56238
  Destination Port: 443
  [Stream index: 158]
  [Stream Packet Number: 30]
  > [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 0]
  Sequence Number: 3711      (relative sequence number)
  Sequence Number (raw): 3549002829
  [Next Sequence Number: 3712      (relative sequence number)]
  Acknowledgment Number: 1535      (relative ack number)
  Acknowledgment number (raw): 3547151597
  0101 .... = Header Length: 20 bytes (5)
v Flags: 0x011 (FIN, ACK)
  000. .... = Reserved: Not set
  ...0 .... = Accurate ECN: Not set
  .... 0... = Congestion Window Reduced: Not set
  .... .0.. = ECN-Echo: Not set
  .... ..0. = Urgent: Not set
  .... ...1 = Acknowledgment: Set
  .... .... 0... = Push: Not set
  .... .... .0.. = Reset: Not set
  .... .... ..0. = Syn: Not set
  > .... .... ...1 = Fin: Set
  > [TCP Flags: .....A...F]
Window: 513
[Calculated window size: 131328]
[Window size scaling factor: 256]
Checksum: 0x1fae [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0

```

第四次挥手：此处没有收到正常挥手，即收到 ACK 确认信号，而是收到 RST 重置连接信号，意味着异常终止。

```
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 10.0.2.15
```

```
Transmission Control Protocol, Src Port: 443, Dst Port: 56238, Seq: 1535, Len: 0
```

```
Source Port: 443
```

```
Destination Port: 56238
```

```
[Stream index: 158]
```

```
[Stream Packet Number: 31]
```

```
> [Conversation completeness: Complete, WITH_DATA (63)]
```

```
[TCP Segment Len: 0]
```

```
Sequence Number: 1535 (relative sequence number)
```

```
Sequence Number (raw): 3547151597
```

```
[Next Sequence Number: 1535 (relative sequence number)]
```

```
Acknowledgment Number: 0
```

```
Acknowledgment number (raw): 0
```

```
0101 .... = Header Length: 20 bytes (5)
```

```
Flags: 0x004 (RST)
```

```
000. .... = Reserved: Not set
```

```
...0 .... = Accurate ECN: Not set
```

```
.... 0... = Congestion Window Reduced: Not set
```

```
.... .0.. = ECN-Echo: Not set
```

```
.... ..0. = Urgent: Not set
```

```
.... ...0 .... = Acknowledgment: Not set
```

```
.... .... 0... = Push: Not set
```

```
> .... .... .1.. = Reset: Set
```

```
.... .... ..0. = Syn: Not set
```

```
.... .... ...0 = Fin: Not set
```

```
[TCP Flags: .....R..]
```

```
Window: 0
```

```
[Calculated window size: 0]
```

```
[Window size scaling factor: 2048]
```

```
Checksum: 0x7193 [unverified]
```

```
[Checksum Status: Unverified]
```

```
Urgent Pointer: 0
```

3、用 Libpcap 或 WinPcap 库侦听网络数据

用 Libpcap 或 WinPcap 库侦听网络上的数据流,解析发送方与接收方的 MAC 和 IP 地址,并作记录与统计。

定义 ip 和 mac 头文件的结构体,定义全局统计变量用于统计;来自不同 MAC 和 IP 地址的通信数据长度。

```
#pragma comment(lib, "Packet")
#pragma comment(lib, "wpcap")
#pragma comment(lib, "WS2_32")
using namespace std;
typedef struct ip_header
{
    u_char ver_ihl;
    u_char tos;
    u_short tlen;
    u_short identification;
    u_short flags_fo;
    u_char ttl;
    u_char proto;
    u_short crc;
    u_char saddr[4];
    u_char daddr[4];
    u_int op_pad;
}ip_header;
typedef struct mac_header
{
    u_char dest_addr[6];
    u_char src_addr[6];
    u_char type[2];
}mac_header;

// 全局统计变量
map<string, long long> src_mac_stats;
map<string, long long> dst_mac_stats;
map<string, long long> src_ip_stats;
map<string, long long> dst_ip_stats;
time_t last_stat_time;
const int STAT_INTERVAL = 60;
```

实现数据抓包函数，将捕捉到的数据写入 scv 文件中，并统计 MAC 和 IP 地址的通信数据长度，每 60s 进行写入。

```

/*prototype of the packet handler*/
void packet_handler(u_char* param, const struct pcap_pkthdr* header, const u_char* pkt_data)
{
    struct tm* ltime;
    char timestr[40];
    ip_header* ih;
    mac_header* mh;
    u_int ip_len;
    time_t local_tv_sec;

    // covert the timestamp to readable format
    local_tv_sec = header->ts.tv_sec;
    strftime(timestr, sizeof timestr, "%Y-%m-%d %H:%M:%S", localtime(&local_tv_sec));

    printf("%s", timestr);
    fprintf(fp, "%s", timestr);

    mh = (mac_header*)pkt_data;
    ih = (ip_header*)(pkt_data + 14);
    // 检查是否为IP数据包
    if (mh->type[0] == 0x08 && mh->type[1] == 0x00)
    {
        format_addr(mh->src_addr, ih->saddr);
        format_addr(mh->dest_addr, ih->daddr);
        printf("%d\n", header->len);
        fprintf(fp, "%d\n", header->len);

        string src_mac = mac_to_string(mh->src_addr);
        string dst_mac = mac_to_string(mh->dest_addr);
        string src_ip = ip_to_string(ih->saddr);
        string dst_ip = ip_to_string(ih->daddr);
        src_mac_stats[src_mac] += header->len;
        dst_mac_stats[dst_mac] += header->len;
        src_ip_stats[src_ip] += header->len;
        dst_ip_stats[dst_ip] += header->len;

        time_t current_time = time(nullptr);
        if (difftime(current_time, last_stat_time) ≥ STAT_INTERVAL)
        {
            write_statics();
            last_stat_time = current_time;
        }
    }
    else
    {
        printf("非IP数据包,,, %d\n", header->len);
        fprintf(fp, "非IP数据包,,, %d\n", header->len);
    }
}

```

main 函数获取设备列表，并根据用户输入对相应的设备进行抓包。


```
int main()
{
    pcap_if_t* alldevs;
    pcap_if_t* d;
    int i = 0;
    char errbuf[PCAP_ERRBUF_SIZE];
    fp = fopen("network_traffic.csv", "w");
    if (!fp)
        exit(0);
    fprintf(fp, "时间,源 MAC,源 IP,目标 MAC,目标 IP,帧长度\n");

    // 获取网络设备列表
    if (pcap_findalldevs(&alldevs, errbuf) == -1) {
        cerr << "获取设备列表失败: " << errbuf << endl;
        return -1;
    }
    // 显示设备列表
    cout << "可用的网络设备:" << endl;
    for (d = alldevs; d != NULL; d = d->next) {
        cout << ++i << ". " << d->name;
        if (d->description)
            cout << " (" << d->description << ")";
        else
            cout << "(No description available)";
        cout << endl;
    }
    if (i == 0) {
        cout << "未找到网络设备!" << endl;
        pcap_freealldevs(alldevs);
        return -1;
    }
}
```

```

// 选择设备
cout << "选择要监听的设备 (1-" << i << "): ";
int inum;
cin >> inum;
if (inum < 1 || inum > i) {
    cout << "选择无效!" << endl;
    pcap_freealldevs(alldevs);
    return -1;
}
// 跳转到选中的设备
for (d = alldevs, i = 0; i < inum - 1; d = d->next, i++);

// 打开网络设备
pcap_t* adhandle;

if ((adhandle = pcap_open(d->name, 65536, 1, 1000, NULL, errbuf)) == NULL) {
    cerr << "打开设备失败: " << errbuf << endl;
    pcap_freealldevs(alldevs);
    return -1;
}

u_int netmask;
struct bpf_program fcode;
char packet_filter[] = "ip";
//预处理
if (pcap_datalink(adhandle) != DLT_EN10MB) {
    fprintf(stderr, "\nThis program works only on Ethernet networks.\n");
    pcap_freealldevs(alldevs);
    return -1;
}
if (d->addresses != NULL)
    netmask = ((struct sockaddr_in*)(d->addresses->netmask))->sin_addr.S_un.S
else
    netmask = 0xffffffff;
//编译过滤器
if (pcap_compile(adhandle, &fcode, packet_filter, 1, netmask) < 0) {
    fprintf(stderr, "\nUnable to compile the packet filter.Check the syntax.\n");
    pcap_freealldevs(alldevs);
    return -1;
}
//设置过滤器
if (pcap_setfilter(adhandle, &fcode) < 0) {
    fprintf(stderr, "\nError setting the filter.\n");
    pcap_freealldevs(alldevs);
    return -1;
}

printf("\nlistening on %s...\n", d->description);
//释放设备列表
pcap_freealldevs(alldevs);
// 开始捕获数据包
pcap_loop(adhandle, 0, packet_handler, NULL);

```

运行结果，监听设备 6 时，可以正常捕获。

```
E:\code-vs\CNhomework\x64\Debug\CNhomework.exe
可用的网络设备:
1. \Device\NPF_{5C5CED45-6CF7-47F4-AC08-61FD6338202A} (Netease UU TAP-Win32 Adapter V9.21)
2. \Device\NPF_{6FE69D95-E6DD-474D-9782-AD9171599123} (TAP-Windows Adapter V9)
3. \Device\NPF_{37D78890-BAE8-42CA-BDB3-7A374FDE1ABA} (Microsoft)
4. \Device\NPF_{AFEC93E7-55F0-4ABF-91A2-04CECBD78414} (Oracle)
5. \Device\NPF_{F660C039-EFBB-47CF-9D62-538579FAF8A0} (VMware Virtual Ethernet Adapter)
6. \Device\NPF_{3972CAFF-3623-41ED-A856-0A98F5962A73} (Microsoft)
7. \Device\NPF_{45113A6A-844E-46CC-BD78-2789BC6A30AA} (VMware Virtual Ethernet Adapter)
8. \Device\NPF_{21234764-7A82-4D00-A02F-E03DBC36DE05} (Microsoft)
选择要监听的设备 (1-8): 6

listening on Microsoft...
2025-10-23 01:59:20, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 36.141.40.31, 55
2025-10-23 01:59:20, c4:c0:63:03:e6:01, 36.141.40.31, f4:6a:dd:ee:2c:4f, 10.27.193.99, 66
2025-10-23 01:59:23, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 106.13.246.66, 191
2025-10-23 01:59:23, c4:c0:63:03:e6:01, 106.13.246.66, f4:6a:dd:ee:2c:4f, 10.27.193.99, 185
2025-10-23 01:59:23, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 106.13.246.66, 54
2025-10-23 01:59:24, d0:39:57:36:3c:d3, 0.0.0.0, ff:ff:ff:ff:ff:ff, 255.255.255.255, 363
2025-10-23 01:59:24, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 223.5.5.81
2025-10-23 01:59:25, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 8.8.8.8, 81
2025-10-23 01:59:25, c4:c0:63:03:e6:01, 8.8.8.8, f4:6a:dd:ee:2c:4f, 10.27.193.99, 117
2025-10-23 01:59:26, c4:c0:63:03:e6:01, 122.248.50.157, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 01:59:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 122.248.50.157, 54
2025-10-23 01:59:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 8.140.245.29, 55
2025-10-23 01:59:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 180.184.93.15, 55
2025-10-23 01:59:26, c4:c0:63:03:e6:01, 111.6.201.96, f4:6a:dd:ee:2c:4f, 10.27.193.99, 79
2025-10-23 01:59:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 111.6.201.96, 83
2025-10-23 01:59:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 180.184.93.15, 819
2025-10-23 01:59:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 8.140.245.29, 852
2025-10-23 01:59:26, c4:c0:63:03:e6:01, 8.140.245.29, f4:6a:dd:ee:2c:4f, 10.27.193.99, 66
```

创建的 network_traffic.csv 如下。

时间	源 MAC	源 IP	目标 MAC	目标 IP	帧长度
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(36.141.40.		55
2025/10/23 1:59	c4:c0:63:(36.141.40.		f4:6a:dd:ε	10.27.193.	66
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(106.13.246		191
2025/10/23 1:59	c4:c0:63:(106.13.246		f4:6a:dd:ε	10.27.193.	185
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(106.13.246		54
2025/10/23 1:59	d0:39:57:ε	0.0.0.0	ff:ff:ff:f	255.255.25	363
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(223.5.5.5		81
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(8.8.8.8		81
2025/10/23 1:59	c4:c0:63:(8.8.8.8		f4:6a:dd:ε	10.27.193.	117
2025/10/23 1:59	c4:c0:63:(122.248.50		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(122.248.50		54
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(8.140.245.		55
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(180.184.93		55
2025/10/23 1:59	c4:c0:63:(111.6.201.		f4:6a:dd:ε	10.27.193.	79
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(111.6.201.		83
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(180.184.93		819
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(8.140.245.		852
2025/10/23 1:59	c4:c0:63:(8.140.245.		f4:6a:dd:ε	10.27.193.	66
2025/10/23 1:59	c4:c0:63:(111.6.201.		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	c4:c0:63:(180.184.93		f4:6a:dd:ε	10.27.193.	66
2025/10/23 1:59	c4:c0:63:(8.140.245.		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	c4:c0:63:(8.140.245.		f4:6a:dd:ε	10.27.193.	781
2025/10/23 1:59	c4:c0:63:(8.140.245.		f4:6a:dd:ε	10.27.193.	88
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(8.140.245.		54
2025/10/23 1:59	c4:c0:63:(180.184.93		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	c4:c0:63:(180.184.93		f4:6a:dd:ε	10.27.193.	724
2025/10/23 1:59	c4:c0:63:(180.184.93		f4:6a:dd:ε	10.27.193.	103
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(180.184.93		54
2025/10/23 1:59	c4:c0:63:(125.78.88.		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	c4:c0:63:(125.78.88.		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(125.78.88.		54
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(125.78.88.		62
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(125.78.88.		54
2025/10/23 1:59	c4:c0:63:(125.78.88.		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	c4:c0:63:(125.78.88.		f4:6a:dd:ε	10.27.193.	60
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(162.159.20		90
2025/10/23 1:59	c4:c0:63:(162.159.20		f4:6a:dd:ε	10.27.193.	90
2025/10/23 1:59	f4:6a:dd:ε	10.27.193.	c4:c0:63:(121.194.11		55
2025/10/23 1:59	c4:c0:63:(121.194.11		f4:6a:dd:ε	10.27.193.	66
2025/10/23 1:59	c4:c0:63:(121.194.11		f4:6a:dd:ε	10.27.193.	100

创建的 `traffic_statistics.csv` 如下。

	A	B	C	D	E	F	
1							
2	统计时间: 2025-10-23 01:59:20						
3	源MAC地址统计:						
4	MAC地址	总数据长度(字节)					
5	f4:6a:dd:e	55					
6	目标MAC地址统计:						
7	MAC地址	总数据长度(字节)					
8	c4:c0:63:0	55					
9	源IP地址统计:						
10	IP地址	总数据长度(字节)					
11	10.27.193.	55					
12	目标IP地址统计:						
13	IP地址	总数据长度(字节)					
14	36.141.40.	55					
15							
16	统计时间: 2025-10-23 02:00:20						
17	源MAC地址统计:						
18	MAC地址	总数据长度(字节)					
19	16:9c:60:9	532					
20	82:09:ea:6	628					
21	bc:03:58:e	1728					
22	c4:c0:63:0	62462					
23	c6:98:bf:b	506					
24	c8:5e:a9:3	1301					
25	ca:59:d8:6	139					
26	d0:39:57:3	1089					
27	de:b5:4f:9	264					
28	e8:c8:29:b	960					
29	f4:6a:dd:e	70339					
30	f8:54:f6:9	162					
31	目标MAC地址统计:						
32	MAC地址	总数据长度(字节)					
33	01:00:5e:0	6220					
34	c4:c0:63:0	70096					
35	f4:6a:dd:e	62462					
36	ff:ff:ff:f	1332					
37	源IP地址统计:						
38	IP地址	总数据长度(字节)					
39	0.0.0.0	1089					
40	1.12.12.21	3072					

4、解析侦听到的网络数据

用 Wireshark 侦听并观察 FTP 数据，分析其用户名密码所在报文的上下文特征，再总结出提取用户名密码的有效方法。解析协议内容，并作记录与统计。对用户登录行为进行记录。

通过 Wireshark 侦听 FTP 数据，其用户名在“USER”之后，用户密码在“PASS”之后，“530”表示登录失败，“230”表示登录成功。

80	5.161813	121.192.180.236	10.27.193.99	FTP	103 Response: 220 Serv-U FTP Server v6.2 for WinSock ready...
82	5.161899	10.27.193.99	121.192.180.236	FTP	70 Request: USER anonymous
83	5.163359	121.192.180.236	10.27.193.99	FTP	124 Response: 331 User name okay, please send complete E-mail address as password.
85	5.163412	10.27.193.99	121.192.180.236	FTP	68 Request: PASS IUSER@
86	5.164786	121.192.180.236	10.27.193.99	FTP	95 Response: 530 Sorry, no ANONYMOUS access allowed.
166	14.956696	121.192.180.236	10.27.193.99	FTP	103 Response: 220 Serv-U FTP Server v6.2 for WinSock ready...
168	14.956826	10.27.193.99	121.192.180.236	FTP	68 Request: USER student
169	14.958382	121.192.180.236	10.27.193.99	FTP	90 Response: 331 User name okay, need password.
171	14.958425	10.27.193.99	121.192.180.236	FTP	76 Request: PASS ILOVESoftware!
172	14.958824	121.192.180.236	10.27.193.99	FTP	74 Response: 530 Not logged in.
211	19.966196	121.192.180.236	10.27.193.99	FTP	103 Response: 220 Serv-U FTP Server v6.2 for WinSock ready...
213	19.966324	10.27.193.99	121.192.180.236	FTP	68 Request: USER student
214	19.967882	121.192.180.236	10.27.193.99	FTP	90 Response: 331 User name okay, need password.
216	19.968837	10.27.193.99	121.192.180.236	FTP	75 Request: PASS ILOVESoftware!
217	19.969374	121.192.180.236	10.27.193.99	FTP	74 Response: 530 Not logged in.
282	25.938172	121.192.180.236	10.27.193.99	FTP	103 Response: 220 Serv-U FTP Server v6.2 for WinSock ready...
284	25.938325	10.27.193.99	121.192.180.236	FTP	68 Request: USER student
285	25.939780	121.192.180.236	10.27.193.99	FTP	90 Response: 331 User name okay, need password.
287	25.939861	10.27.193.99	121.192.180.236	FTP	69 Request: PASS software
288	25.941277	121.192.180.236	10.27.193.99	FTP	74 Response: 530 Not logged in.
386	36.384950	121.192.180.236	10.27.193.99	FTP	103 Response: 220 Serv-U FTP Server v6.2 for WinSock ready...
388	36.385091	10.27.193.99	121.192.180.236	FTP	68 Request: USER student
389	36.386646	121.192.180.236	10.27.193.99	FTP	90 Response: 331 User name okay, need password.
391	36.386735	10.27.193.99	121.192.180.236	FTP	75 Request: PASS ILOVESoftware!
392	36.388306	121.192.180.236	10.27.193.99	FTP	74 Response: 530 Not logged in.
631	58.045854	121.192.180.236	10.27.193.99	FTP	103 Response: 220 Serv-U FTP Server v6.2 for WinSock ready...
633	58.045937	10.27.193.99	121.192.180.236	FTP	68 Request: USER student
634	58.047487	121.192.180.236	10.27.193.99	FTP	90 Response: 331 User name okay, need password.
636	58.047538	10.27.193.99	121.192.180.236	FTP	75 Request: PASS ILOVESoftware!
637	58.050157	121.192.180.236	10.27.193.99	FTP	84 Response: 230 User logged in, proceed.
639	58.050225	10.27.193.99	121.192.180.236	FTP	68 Request: opts utf8 on
640	58.051593	121.192.180.236	10.27.193.99	FTP	75 Response: 501 Invalid option.
642	58.051687	10.27.193.99	121.192.180.236	FTP	60 Request: syst
643	58.053070	121.192.180.236	10.27.193.99	FTP	73 Response: 256 MBV Times: 18

增加 tcp 头文件和 FTP 会话状态的结构体。

```
typedef struct tcp_header
{
    u_short sport;
    u_short dport;
    u_int seq;
    u_int ack;
    u_char th_off;
    u_char flags;
    u_short win;
    u_short sum;
    u_short urp;
}tcp_header;
struct FTPSession
{
    string client_ip;
    string server_ip;
    string username;
    string password;
    string status; //"PENDING" "SUCCESS" "FAILED"
    time_t start_time;
};
```

解析 FTP 协议中的内容，根据用户名在“USER”之后，口令在“PASS”之后，“530”表示登录失败，“230”表示登录成功的规则分别处理 FTP 命令与响应。

```
void parse_ftp_protocol(const u_char* payload, int payload_len, const string& src_ip, const string& dst_ip, const string& src_mac, const string& dst_mac, u_short src_port, u_short dst_port) {
    string data((char*)payload, payload_len);
    string session_key;
    if (dst_port == 21) { // 客户端到服务器
        session_key = src_ip + ":" + to_string(src_port);
    }
    else if (src_port == 21) { // 服务器到客户端
        session_key = dst_ip + ":" + to_string(dst_port);
    }
    else {
        return; // 不是FTP控制连接
    }
    // 处理FTP命令 (客户端->服务器)
    if (dst_port == 21) {
        if (data.find("USER ") == 0) {
            string username = data.substr(5);
            size_t pos = username.find_first_of("\r\n");
            if (pos != string::npos) {
                username = username.substr(0, pos);
            }
            ftp_sessions[session_key] = {
                src_ip, dst_ip, username, "", "PENDING", time(nullptr)
            };
            cout << "检测到FTP登录尝试 - 用户名: " << username << endl;
        }
        else if (data.find("PASS ") == 0) {
            string password = data.substr(5);
            size_t pos = password.find_first_of("\r\n");
            if (pos != string::npos) {
                password = password.substr(0, pos);
            }
            if (ftp_sessions.find(session_key) != ftp_sessions.end()) {
                ftp_sessions[session_key].password = password;
                cout << "检测到FTP密码 - 会话: " << session_key << endl;
            }
        }
    }
}

// 处理FTP响应 (服务器->客户端)
else if (src_port == 21) {
    if (ftp_sessions.find(session_key) != ftp_sessions.end()) {
        FTPSession& session = ftp_sessions[session_key];
        if (data.find("230") == 0) {
            session.status = "SUCCEED";
            string timestamp = get_current_time();
            fprintf(ftp_log_fp, "%s,%s,%s,%s,%s,%s,%s,%s,%s\n", timestamp.c_str(), src_mac.c_str(), src_ip.c_str(), dst_mac.c_str(), dst_ip.c_str(), session.username.c_str(), session.password.c_str(), session.status.c_str());
            cout << "FTP登录成功: " << session.username << endl;
            ftp_sessions.erase(session_key);
        }
        else if (data.find("530") == 0) {
            session.status = "FAILED";
            string timestamp = get_current_time();
            fprintf(ftp_log_fp, "%s,%s,%s,%s,%s,%s,%s,%s,%s\n", timestamp.c_str(), src_mac.c_str(), src_ip.c_str(), dst_mac.c_str(), dst_ip.c_str(), session.username.c_str(), session.password.c_str(), session.status.c_str());
            cout << "FTP登录失败: " << session.username << endl;
            ftp_sessions.erase(session_key);
        }
    }
}
```

实现数据抓包函数，与第 3 项实验要求类似，新增对 FTP 协议解析函数的调用。


```

void packet_handler(u_char* param, const struct pcap_pkthdr* header, const u_char* pkt_data)
{
    char timestr[40];
    u_char* payload;
    ip_header* ih;
    mac_header* mh;
    time_t local_tv_sec;

    local_tv_sec = header->ts.tv_sec;
    strftime(timestr, sizeof(timestr), "%Y-%m-%d %H:%M:%S", localtime(&local_tv_sec));

    mh = (mac_header*)pkt_data;
    //ih = (ip_header*)(pkt_data + sizeof(mac_header));
    if (mh->type[0] == 0x08 && mh->type[1] == 0x00) {
        ip_header* ih = (ip_header*)(pkt_data + 14);

        string src_mac = mac_to_string(mh->src_addr);
        string dst_mac = mac_to_string(mh->dest_addr);
        string src_ip = ip_to_string(ih->saddr);
        string dst_ip = ip_to_string(ih->daddr);

        printf("%s,", timestr);
        fprintf(fp, "%s,", timestr);
        printf("%s,%s,%s,%s,%d\n", src_mac.c_str(), src_ip.c_str(), dst_mac.c_str(), dst_ip.c_str(), header->len);
        fprintf(fp, "%s,%s,%s,%s,%d\n", src_mac.c_str(), src_ip.c_str(), dst_mac.c_str(), dst_ip.c_str(), header->len);

        src_mac_stats[src_mac] += header->len;
        dst_mac_stats[dst_mac] += header->len;
        src_ip_stats[src_ip] += header->len;
        dst_ip_stats[dst_ip] += header->len;

        // 检查是否为TCP协议
        if (ih->proto == 6) {
            tcp_header* th = (tcp_header*)(pkt_data + 14 + (ih->ver_ihl & 0x0F) * 4);
            u_short src_port = ntohs(th->sport);
            u_short dst_port = ntohs(th->dport);

            int ip_header_len = (ih->ver_ihl & 0x0F) * 4;
            int tcp_header_len = (th->th_off >> 4) * 4;
            int payload_len = header->len - 14 - ip_header_len - tcp_header_len;

            if (payload_len > 0) {
                const u_char* payload = pkt_data + 14 + ip_header_len + tcp_header_len;

                // 检查是否为FTP流量 (端口21)
                if (src_port == 21 || dst_port == 21) {
                    parse_ftp_protocol(payload, payload_len, src_ip, dst_ip, src_mac, dst_mac, src_port, dst_port);
                }
            }
        }

        time_t current_time = time(nullptr);
        if (difftime(current_time, last_stat_time) ≥ STAT_INTERVAL) {
            write_statistics();
            last_stat_time = current_time;
        }
    }
}

```

main 函数新增存储用户登录行为的日志表。

```

ftp_log_fp = fopen("ftp_logins.csv", "w");
if (!ftp_log_fp)
    exit(0);
fprintf(ftp_log_fp, "时间,源MAC,源IP,目标MAC,目标IP,登录名,口令,成功与否\n");

```

运行结果，在监听过程中，尝试登录 ftp 服务器，一次输入失败的密码，一次输入成功的密码。

```

E:\code-vs\CNhomework\64\Debug\CNhomework.exe
1. \Device\NPF_{5C5CED45-6CF7-47F4-AC08-61FD6338202A} (Netease UU TAP-Win32 Adapter V9.21)
2. \Device\NPF_{6FE69D95-E6DD-474D-9782-AD9171599123} (TAP-Windows Adapter V9)
3. \Device\NPF_{37D78890-BAE8-42CA-BDB3-7A374FDE1ABA} (Microsoft)
4. \Device\NPF_{AFEC93E7-55F0-4ABF-91A2-04CECDB78414} (Oracle)
5. \Device\NPF_{F660C039-EFBB-47CF-9D62-538579FAF8A0} (VMware Virtual Ethernet Adapter)
6. \Device\NPF_{3972CAFF-3623-41ED-A856-0A98F5962A73} (Microsoft)
7. \Device\NPF_{45113A6A-844E-46CC-BD78-2789BC6A30AA} (VMware Virtual Ethernet Adapter)
8. \Device\NPF_{21234764-7A82-4D00-A02F-E03DBC36DE05} (Microsoft)
Enter the interface number (1-8):6

listening on Microsoft...
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 66
2025-10-23 04:06:11, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 66
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:11, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 103
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 70
检测到FTP登录尝试 - 用户名: anonymous
2025-10-23 04:06:11, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 124
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 68
检测到FTP密码 - 会话: 10.27.193.99:60702
2025-10-23 04:06:11, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 95
FTP登录失败: anonymous
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:11, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 04:06:11, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 04:06:11, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:19, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 66
2025-10-23 04:06:19, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:19, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 04:06:19, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 04:06:19, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 66
2025-10-23 04:06:26, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 66
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:26, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 103
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 68
检测到FTP登录尝试 - 用户名: student
2025-10-23 04:06:26, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 90
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 75
检测到FTP密码 - 会话: 10.27.193.99:60750
2025-10-23 04:06:26, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 84
FTP登录成功: student
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 68
2025-10-23 04:06:26, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 75
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 59
2025-10-23 04:06:26, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 85
2025-10-23 04:06:26, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:27, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54
2025-10-23 04:06:27, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 04:06:27, c4:c0:63:03:e6:01, 121.192.180.236, f4:6a:dd:ee:2c:4f, 10.27.193.99, 60
2025-10-23 04:06:27, f4:6a:dd:ee:2c:4f, 10.27.193.99, c4:c0:63:03:e6:01, 121.192.180.236, 54

```

创建的 traffic_statistics_ftp.csv 如下。

	A	B	C
1			
2	统计时间: 2025-10-23 04:06:12		
3	源MAC地址统计:		
4	MAC地址	总数据长度(字节)	
5	f4:6a:dd:e	66	
6	目标MAC地址统计:		
7	MAC地址	总数据长度(字节)	
8	c4:c0:63:c	66	
9	源IP地址统计:		
10	IP地址	总数据长度(字节)	
11	10. 27. 193.	66	
12	目标IP地址统计:		
13	IP地址	总数据长度(字节)	
14	121. 192. 18	66	
15			

创建的 ftp_logins.csv 如下。

时间	源MAC	源IP	目标MAC	目标IP	登录名	口令	成功与否
2025/10/23 4:06	c4:c0:63:c	121. 192. 18	f4:6a:dd:e	10. 27. 193.	anonymous	IEUser@	FAILED
2025/10/23 4:06	c4:c0:63:c	121. 192. 18	f4:6a:dd:e	10. 27. 193.	student	software	FAILED
2025/10/23 4:06	c4:c0:63:c	121. 192. 18	f4:6a:dd:e	10. 27. 193.	student	ILoveSoftw	SUCCEED

创建的 network_traffic_ftp.csv 如下。

	A	B	C	D	E	F	G
1	时间	源MAC	源IP	目标MAC	目标IP	帧长度	
2	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			66	
3	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			66	
4	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
5	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			103	
6	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
7	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			70	
8	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			124	
9	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
10	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			68	
11	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			95	
12	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
13	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
14	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			60	
15	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			60	
16	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
17	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			66	
18	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			66	
19	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
20	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			103	
21	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
22	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			68	
23	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			90	
24	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
25	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			69	
26	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			74	
27	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
28	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
29	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			60	
30	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			60	
31	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
32	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			66	
33	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			66	
34	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
35	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			103	
36	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
37	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			68	
38	2025/10/23 4:06	c4:c0:63:0121.192.18	f4:6a:dd:ε10.27.193.			90	
39	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			54	
40	2025/10/23 4:06	f4:6a:dd:ε10.27.193.	c4:c0:63:0121.192.18			75	

4 实验代码

本次实验的代码已上传于以下代码仓库：<https://gitee.com/koshistar/cni-exp>。

5 实验总结

本次实验内容超出了当前所学内容，学到了许多之后的知识，同时，初步熟悉了 Wireshark 的和 Winpcap 包的使用。

本次实验参考了学长的实验报告，并利用网络内容和 AI 进行学习并改进。

以下是遇到的一些问题。

(1) winpcap 包的引入，`#include <pcap.h>`，`#include <Packet32.h>`，`#include <ntddndis.h>`需要放在其他 `include` 之后，否则会出现 `inline` 的重载报错。

(2) 实验要求 4 需要以管理员身份运行。

(3) 本机上有 8 个可监听设备，只有监听 6 时能在 Windows 环境下捕获到内容。

(4) 尝试使用 FileZilla 搭建 FTP 服务器，但是单机的情况下未能捕获到 FTP 协议，利用软工 FTP 的服务器完成了实验。